

Solutions Architect — Design Concerns Handbook

The 24 things an architect reasons about, the AWS services that address each, the trade-offs, and the questions to ask.

How to use this for an interview. An AWS SA interview rarely asks "what does service X do". It gives you a scenario and watches how you reason: what you ask before designing, what you choose, and — most revealingly — **what you rule out and why**. Every section below has a **green box of questions** (what to ask the customer) and an **orange box of trade-offs** (what you give up). Those two boxes are what interviewers are actually listening for.

Also worth knowing: AWS interviews are heavily weighted on **Leadership Principles**, assessed through behavioural questions in every round. Technical depth alone does not carry an AWS loop.

Part 1 — Foundations

1. Requirements & Constraints

Everything else derives from here. Architects who start drawing boxes before establishing constraints are the ones who get filtered out.

- **Functional** — what must it do
- **Non-functional** — how fast, how available, how many users, how much data
- **Constraints** — budget, timeline, existing systems, team skills, regulation
- **Success criteria** — how will we know it worked

Ask: Who are the users and how many? What is the peak versus average load? What is the acceptable response time? What happens to the business if this is down for an hour? What must this integrate with? What regulations apply? What is the budget envelope? What is the timeline?

Trade-off: gathering requirements costs time and can look slow. Not gathering them costs a rebuild. In an interview, spending the first three minutes asking questions is a signal of seniority, not hesitation.

2. Compute

Option	Choose when	Give up
Lambda	Event-driven, spiky, short tasks; no servers to manage	15-min ceiling, cold starts, runtime constraints
Fargate	Containers without managing instances	Cost per task above steady high utilisation
ECS / EKS on EC2	Steady load, cost control, GPU or specialised instances	You manage capacity and patching
EC2	Full control, licensing constraints, lift-and-shift	Everything is your responsibility

Also: Auto Scaling (target tracking / step / scheduled / predictive), Spot for interruptible work, Savings Plans and Reserved Instances for committed baseline.

Ask: Is the load steady or spiky? What is the longest a single unit of work runs? Does the team already run containers? Any licensing tied to cores? Can any workload tolerate interruption?

Trade-off: serverless trades cost-at-scale and control for operational simplicity. The crossover point is real — a busy service can be cheaper on EC2 with Savings Plans than on Lambda. Say so when it applies.

3. Storage

Option	Nature	Choose when
S3	Object	Files, backups, data lake, static content, anything unstructured
EBS	Block, single-instance	Boot volumes, databases on EC2
EFS	File, shared, multi-AZ	Many instances need the same filesystem
FSx	File, specialised	Windows (SMB), Lustre (HPC), NetApp, OpenZFS

S3 depth: storage classes and Intelligent-Tiering; lifecycle transitions and expiry; versioning; Object Lock (WORM, compliance); replication (CRR/SRR); encryption (SSE-S3, SSE-KMS, SSE-C); event notifications; presigned URLs.

Ask: How much data, and how fast is it growing? What is the access pattern — hot, warm, or write-once-read-never? How long must it be retained, and is that a legal requirement? Does anything need to be immutable?

Trade-off: cheaper storage classes cost more to retrieve and impose minimum durations. Intelligent-Tiering removes the guesswork at a small per-object monitoring fee — usually the right default when access patterns are unknown.

4. Databases

Engine	Choose when
RDS / Aurora	Relational — joins, transactions, ad-hoc queries. Aurora for performance and storage auto-scaling; Aurora Serverless v2 for variable load.
DynamoDB	Known access patterns, extreme scale, single-digit ms. Design the key schema around the queries, not the entities.
ElastiCache	Sub-ms reads on hot data; session store; Redis for data structures, Memcached for simple caching
OpenSearch	Full-text search, log analytics, vector search
Neptune / Timestream / DocumentDB / Keyspaces	Graph / time-series / document / wide-column

Ask: What are the top five queries this system will run? Read-heavy or write-heavy? Does it need transactions across entities? What consistency does the business actually require? How large will it get?

Trade-off: DynamoDB gives unlimited scale in exchange for **designing around fixed access patterns** — a new query pattern later can mean a new GSI or a migration. Relational gives query flexibility and pays for it in scaling effort. **Purpose-built beats one-size-fits-all**, but every extra engine is another thing to operate.

Part 2 — Networking & Connectivity

5. Network Design

- **VPC and CIDR planning** — leave room; overlapping CIDRs block future peering
- **Subnets across AZs** — public for load balancers, private for application and data
- **Security groups** (stateful, resource-level, allow-only) vs **NACLs** (stateless, subnet-level, allow and deny)
- **Egress control** — NAT Gateway for outbound; VPC endpoints to keep AWS traffic off the internet; Network Firewall for domain filtering

Ask: What must be reachable from the internet, and what must never be? Are there existing CIDR ranges to avoid? Is there a requirement that traffic never traverses the public internet? Who else needs to reach this?

Trade-off: NAT Gateways cost per hour *and* per GB — heavy S3 traffic through NAT is a common and avoidable bill. A gateway endpoint for S3 is free and removes it entirely.

6. Connectivity & Hybrid

Need	Service
VPC to VPC, few of them	VPC Peering (non-transitive)
Many VPCs, transitive routing	Transit Gateway — route tables give domain isolation; black hole routes deliberately drop traffic
On-prem, encrypted, quick to set up	Site-to-Site VPN
On-prem, consistent bandwidth and latency	Direct Connect — BGP policies can prioritise traffic types
Expose your service privately to others	PrivateLink
Data cannot leave the premises	Outposts

Ask: How much data crosses between environments, and how often? What latency does the on-prem integration need? Is a VPN acceptable or is dedicated bandwidth required? How many accounts and VPCs will there eventually be?

Trade-off: Direct Connect takes weeks to provision and costs a fixed monthly fee. VPN is available today at a fraction of the cost with less predictable latency. A common answer is **VPN now, Direct Connect later, with VPN as the backup path.**

7. Traffic Routing & Content Delivery

Service	Use for
Route 53	DNS with routing policies: simple, weighted, latency, failover, geolocation, geoproximity, multi-value. Health checks drive failover.
CloudFront	Caching static and cacheable dynamic content at the edge; also the place to attach WAF
Global Accelerator	Static anycast IPs, AWS backbone transit, failover in seconds rather than DNS TTL ; for non-cacheable traffic
ALB / NLB	Regional distribution — ALB for HTTP routing rules, NLB for TCP, static IPs and extreme throughput

Ask: Where are the users? Is the content cacheable? How quickly must failover happen? Do partners need to allowlist fixed IPs?

Trade-off: DNS-based failover is cheap but bounded by TTL and client caching. Global Accelerator is faster and costs a fixed hourly fee plus data transfer. Choose on **how much a minute of downtime costs.**

Part 3 — Security, Identity & Compliance

8. Identity & Access

- **Authentication** (who are you) vs **authorization** (what may you do) — keep them distinct in your answer
- **IAM roles over users**; temporary credentials over long-lived keys, always
- **Policy types**: identity-based, resource-based, permission boundaries, SCPs, session policies. **Explicit deny always wins.**
- **RBAC vs ABAC** — ABAC uses tags, so new cases don't require new policies
- **Federation**: IAM Identity Center for workforce, Cognito for customers

Ask: Who needs access, and are they employees or customers? Is there an existing identity provider? What is the granularity — service level, or per-record? Who approves access changes? How are permissions reviewed?

Trade-off: least privilege is more work upfront and generates access requests later. Broad permissions are fast and become the finding in your first audit. **Permission boundaries** are the usual compromise: let teams self-serve within a ceiling.

9. Data Protection

- **In transit** — TLS everywhere; ACM for certificates; private paths via VPC endpoints
- **At rest** — KMS. Customer-managed keys when you need the key policy and audit trail; multi-Region keys when data is processed in several regions; grants for temporary fine-grained access
- **Secrets** — Secrets Manager when rotation is required; Parameter Store when it is not
- **Classification and discovery** — Macie finds sensitive data already sitting in S3
- **Techniques** — masking, tokenization, pseudonymization, format-preserving encryption; each differs in reversibility and how much context survives

Ask: What data classifications exist? Is any of it PII, PHI or cardholder data? Who may see raw versus masked data? Are there key custody requirements? What must happen on a deletion request?

Trade-off: customer-managed KMS keys give control and auditability at a per-key cost and per-request charge, plus the operational burden of key policies. Encryption also complicates cross-account sharing — the key policy must permit it too, which is a frequent cause of confusing access denials.

10. Compliance & Governance

- **Multi-account strategy** — Organizations with OUs; separate accounts are the strongest blast-radius boundary AWS offers
- **SCPs** set a ceiling and **never grant**; not even an account admin can exceed them
- **Control Tower** — landing zone with preventive and detective controls; blocks non-compliant deployments before they happen
- **CloudTrail** for the audit trail (data events are off by default); **Config** for resource state and compliance rules; **Security Hub** to aggregate findings; **Audit Manager** to automate evidence collection
- **Data residency** — region selection, replication boundaries, and controls that enforce them

Ask: Which regulations apply, and in which jurisdictions? What must be provable to an auditor, and how often? What is the account structure today? Who owns security policy versus who implements it?

Trade-off: more accounts give better isolation and cost attribution, at the price of more networking, more identity plumbing, and more automation to keep consistent. Under-segmenting is a security problem; over-segmenting is an operations problem.

Part 4 — Reliability

11. Availability

- **Multi-AZ is the baseline** for anything production. Multi-region is a much larger commitment.
- Availability targets compound: 99.9% is ~43 minutes of downtime a month; 99.99% is ~4 minutes. **Each nine costs disproportionately more.**
- Every dependency reduces your ceiling — your availability cannot exceed the product of what you depend on
- Health checks, automated replacement, and load balancer draining

Ask: What does an hour of downtime cost? Is there an SLA to customers, and what are the penalties? Is degraded service acceptable, or is it all-or-nothing? Which dependencies are outside our control?

Trade-off: multi-region active-active roughly doubles cost and adds data-consistency problems that are genuinely hard. **Push back on "we need multi-region" until you know what downtime actually costs** — that question is usually the most valuable thing you contribute.

12. Resilience & Failure Handling

- **Identify failure modes** — instance, AZ, region, dependency, and the poisoned deployment
- **Blast radius** — cell-based architecture, shuffle sharding, bulkheads to contain failure
- **Retry with exponential backoff and jitter** for transient failures; **circuit breakers** for sustained ones
- **Graceful degradation** — reduced functionality beats an error page
- **Timeouts at every layer**, and they must be consistent — an inner timeout longer than the outer one produces confusing failures
- **Idempotency** — retries will happen; duplicate side effects must not

Ask: What is the single most damaging thing that could fail? What should users see when a dependency is down? Are any operations non-idempotent — payments, emails, provisioning? How is the system tested against failure?

Trade-off: retries without jitter and circuit breakers can amplify an incident — every client retrying in unison is a self-inflicted DDoS. **Naming this shows real operational experience.**

13. Disaster Recovery

Strategy	RTO / RPO	Cost
Backup & restore	Hours	Lowest
Pilot light	Tens of minutes	Low — core systems idle
Warm standby	Minutes	Medium — scaled-down live copy
Multi-site active-active	Near zero	Highest

Ask: What is the RTO — how long can we be down? What is the RPO — how much data can we lose? Has DR ever been tested? Who declares a disaster, and how is failback handled?

Trade-off: DR is insurance — you are buying down risk with money. **An untested DR plan does not exist.** If the customer has never run a failover, that is the first recommendation, ahead of any architecture change.

Part 5 — Performance & Scale

14. Latency & Response Time

- Where the time goes: network, compute, data access, and any downstream dependency
- **Measure percentiles, not averages** — p50 describes the typical user, p99 describes your worst experience and your support tickets
- Levers: caching, edge delivery, region placement, connection reuse, protocol choice, precomputation
- **Perceived latency** is a separate lever — streaming, skeleton screens and progress indicators change the experience without changing the number

Ask: What latency do users actually notice? Is this measured at the server or in the browser? Which percentile matters to the business? Where are the users relative to the workload?

Trade-off: low latency usually means provisioned, always-warm capacity — the opposite of cost efficiency. Caching improves latency and cost together but introduces staleness, so the real question becomes **how stale is acceptable**.

15. Scalability & Throughput

- **Vertical** (bigger) vs **horizontal** (more) — horizontal is the AWS default and requires statelessness
- **Where the bottleneck actually is** — usually the database or a downstream limit, rarely the application tier
- **Service quotas and downstream limits** — check them; they cause more incidents than capacity does
- Scaling patterns: target tracking, step, scheduled, predictive; **queues absorb spikes so you provision for average rather than peak**

Ask: What is the peak-to-average ratio? How fast does load ramp — seconds or hours? What is the growth projection? Which component saturates first? Are there known quota limits?

Trade-off: reactive auto scaling always lags — by the time it reacts, the spike may be over. Scheduled or predictive scaling handles known patterns; queuing handles unknown ones at the cost of latency.

16. Caching

Layer	Service
Edge	CloudFront
API	API Gateway cache
Application	ElastiCache (Redis / Memcached)
Database	Read replicas, DAX for DynamoDB

Ask: What proportion of requests are repeated? How stale can data be? What must invalidate the cache? Is the content personalised — because that limits edge caching sharply?

Trade-off: caching is the highest-leverage optimisation available and brings **the hardest correctness problem in computing — invalidation**. Measure hit rate; a cache with a 5% hit rate costs money and adds a failure mode for nothing.

17. State Management

- **Stateless compute** is what makes horizontal scaling and instance replacement possible
- Externalise session state — ElastiCache, DynamoDB, or a token the client carries
- **Consistency** — strong versus eventual, and which one the business actually needs
- Sticky sessions work but weaken scaling and failure recovery; treat them as a compromise, not a design

Ask: What must survive an instance being replaced mid-request? Can a user's next request land on a different node? Where does eventual consistency become a business problem?

18. Data Ingestion & Movement

Pattern	Services
Batch / scheduled	Glue, Batch, EMR, DataSync, Transfer Family
Streaming	Kinesis Data Streams / Firehose, MSK
Change data capture	DMS, DynamoDB Streams, database triggers
SaaS sources	AppFlow
Bulk physical transfer	Snowball / Snowmobile

Ask: How much data, how often, and how fresh must it be? Is the source system able to push, or must we pull? What happens if ingestion fails — replay, or is the data gone? Are duplicates possible?

Trade-off: streaming gives freshness and costs continuous infrastructure and more operational complexity. **Ask what freshness the business decision actually requires** — the honest answer is often "hourly is fine", which halves the design.

19. Data Lifecycle

- **Retention** — how long, and is it a legal minimum or a legal maximum? Both exist, and they conflict.
- **Archival** — lifecycle transitions to colder storage classes
- **Deletion** — including deletion requests, and **proving** deletion happened
- **Versioning and immutability** — S3 versioning; Object Lock for WORM compliance
- **Classification** — because retention and access rules depend on it

Ask: What is the required retention period per data type? Is there a maximum retention we must not exceed? Must deletion be provable? Do backups also need to honour deletion requests — because that is where most designs fail?

Trade-off: lifecycle automation is cheap and prevents human error, but a wrong rule silently deletes data. Test rules on a non-production bucket first, and prefer transition-then-expire over immediate expiry.

20. Analytics

Need	Service
Query data in S3	Athena (serverless, per-query)
Data warehouse	Redshift
ETL and cataloguing	Glue, Glue Data Catalog
Big data processing	EMR
Dashboards	QuickSight
Log analytics	OpenSearch, CloudWatch Logs Insights
Fine-grained data lake permissions	Lake Formation

Ask: Who consumes the analytics, and how often? Is this exploratory or a fixed set of reports? How fresh must it be? What is the query volume — because that decides Athena versus Redshift?

Trade-off: Athena costs nothing when idle and charges per TB scanned — excellent for occasional queries, expensive for a dashboard that refreshes every minute. Partitioning and columnar formats (Parquet) change that economics dramatically.

Part 7 — Integration & Operations

21. Integration & Coupling

- **Synchronous** — simple, immediate, but couples availability: your caller is only as available as you are
- **Asynchronous** — decoupled, absorbs spikes, survives downstream outages; costs you immediate confirmation
- Choosing the mechanism: SQS (queue), SNS (fan-out), EventBridge (content routing), Kinesis (ordered replayable stream), Step Functions (orchestration with state)
- **Dead-letter queues** so failures are inspectable rather than lost
- **Idempotency and API versioning** — both are contract concerns, not implementation details

Ask: Does the caller need an answer now, or just an acknowledgement? What happens if the downstream system is down for an hour? Does order matter? Can the same message arrive twice?

Trade-off: async decoupling is almost always the more resilient choice and it makes the system harder to reason about, harder to trace, and harder to test. **Say both halves** — an architect who only names the benefit sounds junior.

22. Deployment & Release

- **Environment separation** — separate accounts are stronger than separate VPCs
- **IaC** — CloudFormation, CDK or Terraform; the environments must be reproducible
- **Pipeline** — CodePipeline / CodeBuild / CodeDeploy, with quality gates that can block a release
- **Strategies** — rolling, blue/green, canary, linear, shadow. Each trades rollback speed against cost and exposure.
- **Rollback must be defined before deploy**, and database migrations are where rollback usually breaks

Ask: How often do you release? What is the rollback procedure, and when was it last exercised? Are schema migrations backward-compatible? Who approves production changes?

Trade-off: blue/green gives instant rollback for double the infrastructure during cutover. Canary limits exposure but takes longer and needs good metrics to decide on. **Neither helps if the database change is not reversible** — mention that.

23. Observability & Operations

- **Three pillars:** metrics (CloudWatch), logs (CloudWatch Logs, Logs Insights), traces (X-Ray)
- **CloudWatch tells you how slow; X-Ray tells you where**
- Alarms on symptoms users feel, not on every metric; composite alarms and anomaly detection reduce noise
- **Synthetics** catch problems before users do; **RUM** measures what users actually experienced
- Correlation IDs across services; structured logging so logs are queryable
- Runbooks, on-call, and blameless post-incident review

Ask: How do you currently find out something is broken — monitoring, or a customer call? What is alerting on now, and how much of it is ignored? Can you trace a single request end to end?

Trade-off: full-fidelity tracing and log retention cost real money at scale. Sampling — low rate for healthy traffic, 100% for errors — is the standard compromise. **Alert fatigue is a genuine failure mode:** too many alarms is functionally the same as none.

24. Cost Optimisation

- **Pricing models** — On-Demand, Savings Plans, Reserved Instances, Spot. Commit to the baseline, burst on-demand, run interruptible work on Spot.
- **Right-sizing** from real utilisation data (Compute Optimizer), not from the original guess
- **Storage tiering** and lifecycle policies
- **Data transfer** — cross-AZ, cross-region and NAT charges are the usual surprises
- **Attribution** — cost allocation tags, because you cannot optimise what you cannot attribute
- **Budgets** for proactive alerting; Cost Explorer for analysis

Ask: What is the current spend and what is it growing at? Which team owns which cost? Is anything running that nobody uses? What is the tagging discipline like?

Trade-off: cost optimisation trades against performance, resilience and engineering time. The best savings are usually structural — **delete the unused, right-size the oversized, and stop paying for idle** — not clever architecture. Say that; it reads as pragmatic rather than dogmatic.

Using This in the Interview

A design question, worked

Phase	What to do
1. Clarify	Ask about users, scale, latency, availability, budget, regulation, existing systems. Do not draw anything yet. Two to four minutes here is normal and expected.
2. Restate	"So the constraints are... and the priorities in order are..." — this confirms you listened and lets them correct you cheaply.
3. High-level design	Boxes and arrows, ingress to data. Keep it simple first; do not name every service.
4. Justify choices	For each significant decision, say what you considered and why you rejected it . This is the part being scored.
5. Deep dive	They will pick something and push. Go as deep as you actually know, and say plainly where your knowledge ends.
6. Failure modes	Volunteer this if they don't ask: "if this component fails, here's what happens and here's the mitigation."
7. Evolution	"At 10× this load, the first thing to break is X, and here's what I'd change."

Phrases that signal seniority

- "Before I design this, can I check what happens to the business if it's down for an hour?"
- "I'd start with the simplest thing that meets the requirement, and here's the trigger that would make me add complexity."
- "That gives us X, but we pay for it with Y. Given the constraint you described, I'd take that trade."
- "I don't know that service well enough to design with it. What I do know is the pattern, which is..."
- "I'd want to validate that assumption with data before committing to it."

Things that read as junior

- Naming services before establishing requirements
- Multi-region by reflex, without asking what downtime costs
- Presenting only the upside of a choice
- Maximum availability, maximum security and minimum cost in the same design, without acknowledging the conflict
- Bluffing on depth rather than saying "I'd need to check that"

Leadership Principles — the ones that surface most in SA loops

Principle	What they're probing
Customer Obsession	Did you start from the customer's problem, or from the technology you wanted to use?
Dive Deep	Can you go three questions deeper than the summary, with specifics?
Are Right, A Lot	Sound judgement under incomplete information — and seeking out other views
Earn Trust	Do you own mistakes plainly? Do you say "I don't know"?
Bias for Action	Reversible decisions made quickly; irreversible ones made carefully
Invent and Simplify	Did you remove complexity, or add it?
Deliver Results	What actually shipped, and what was the measured outcome?

Preparation that works: write six to eight real stories in **STAR** form (Situation, Task, Action, Result) with **numbers in the result**. Each story should map to two or three principles. Include at least one genuine failure — the follow-up question is always "what would you do differently", and a story with no mistake in it invites doubt.

The one-line version of each concern

Concern	The question it really asks
Requirements	What are we actually solving?
Compute	Who manages the servers?
Storage	How is it accessed, and for how long?
Databases	What are the queries?
Network design	What can reach what?
Connectivity	How do the environments meet?
Traffic routing	Which copy serves this user?
Identity	Who, and what may they do?
Data protection	Who can read it if they get in?
Compliance	Can we prove it to an auditor?
Availability	What does downtime cost?
Resilience	What happens when it breaks?
Disaster recovery	How fast back, how much lost?
Latency	Where does the time go?
Scalability	What breaks first at 10×?
Caching	How stale is acceptable?
State	What survives an instance dying?
Ingestion	How fresh must it be?
Data lifecycle	When does it go away, and can we prove it?
Analytics	Who asks what of the data?
Integration	Must the caller wait?
Deployment	How do we undo it?
Observability	How do we know it's broken?
Cost	What are we paying for that nobody uses?

Service limits, pricing and features change. Verify specifics against current AWS documentation.